

Actividad 2.1: Clientes y Año de Nacimiento

Este ejercicio te pide crear una tabla simple pero con una **validación**: que el año de nacimiento sea lógico (entre 1900 y 2025).

Copia y pega esto en tu MySQL:

SQL

```
-- Preparación (Opcional, por si quieres tenerlo ordenado)
DROP DATABASE IF EXISTS actividad2_1;
CREATE DATABASE actividad2_1;
USE actividad2_1;

-- Solución del Ejercicio
CREATE TABLE clientes (
    nombre VARCHAR(50) NOT NULL,
    anyo_nacimiento NUMERIC(4,0) NOT NULL,
    -- Restricción para validar el año
    CONSTRAINT chk_anyo_nacimiento CHECK (anyo_nacimiento >= 1900 AND
anyo_nacimiento <= 2025)
);
```

Pruébalo: Intenta insertar a alguien del año 1800 para ver cómo falla el CHECK.

Actividad 2.2: Planetas (Uso de Auto-incremental)

Enunciado: Diseña una tabla llamada `planetas` donde cada planeta tenga un identificador numérico que se genere solo, un nombre obligatorio y una distancia a la Tierra. Es imposible que la distancia sea negativa o cero.

SQL

```
DROP DATABASE IF EXISTS actividad2_2;
CREATE DATABASE actividad2_2;
USE actividad2_2;

CREATE TABLE planetas (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    distancia_tierra NUMERIC(10,0) NOT NULL,

    CONSTRAINT pk_planetas PRIMARY KEY (id),
    -- Validación lógica: la distancia debe ser positiva
    CONSTRAINT chk_distnacia CHECK (distancia_tierra > 0)
);
```

Fíjate en esto:

1. **AUTO_INCREMENT**: Va siempre junto a la **Primary Key** cuando es un número `INT`. Así no tienes que inventarte los IDs (1, 2, 3...) al insertar datos.
2. **NUMERIC (10, 0)**: Significa 10 dígitos en total y 0 decimales (enteros grandes). Podría usarse `BIGINT` también.

Actividad 2.3: Actores (Restricción UNIQUE)

Enunciado: Crea una tabla de actores con un código automático. El nombre del actor no puede repetirse en la base de datos (para evitar duplicados). Además, la fecha de nacimiento debe ser posterior a 1900.

SQL

```
DROP DATABASE IF EXISTS actividad2_3;
CREATE DATABASE actividad2_3;
USE actividad2_3;

CREATE TABLE actores (
    codigo INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL UNIQUE, -- Ojo aquí
    fecha_nacimiento DATE NOT NULL,

    CONSTRAINT pk_actores PRIMARY KEY (codigo),
    -- Las fechas van entre comillas simples 'AAAA-MM-DD'
    CONSTRAINT chk_fecha_nacimiento CHECK (fecha_nacimiento > '1900-01-01')
);
```

Fíjate en esto:

1. UNIQUE vs PRIMARY KEY:

- La PK es el código (número interno para la BD).
- El nombre es UNIQUE (regla de negocio: no queremos dos "Brad Pitt").

2. Comillas en fechas: En el CHECK, la fecha '1900-01-01' debe ir entre comillas simples, como si fuera texto.

Actividad 2.4: Personajes (La Primera Relación)

Enunciado: Partiendo de la tabla de actores anterior, crea una tabla `personajes`. Cada personaje es interpretado por un actor. Si eliminamos al actor de la base de datos, deben desaparecer automáticamente todos sus personajes (borrado en cascada).

SQL

```
-- (Asumimos que la tabla 'actores' del ejercicio 2.3 ya existe)

CREATE TABLE personajes (
    codigo INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    codigo_actor INT NOT NULL,

    CONSTRAINT pk_personajes PRIMARY KEY (codigo),

    -- Relación con la tabla Actores
    CONSTRAINT fk_personajes_actores FOREIGN KEY (codigo_actor)
        REFERENCES actores(codigo) ON DELETE CASCADE
);
```

Fíjate en esto:

1. **Orden de creación:** Primero tienes que crear actores (Padre) y luego personajes (Hija). Si lo haces al revés, dará error.
2. **ON DELETE CASCADE:** Esta es la instrucción clave. Significa: "Si borro al actor Padre, mata a los personajes Hijos". Sin esto, la base de datos te prohibiría borrar al actor mientras tenga personajes.

Actividad 2.5: Productos y Categorías (Borrado en Cascada)

Enunciado: Vamos a gestionar un inventario. Tienes categorías (ej. "Fruta", "Pescado") y productos que pertenecen a esas categorías. La regla de oro es: si decido dejar de vender una categoría entera (borro "Pescado"), quiero que **automáticamente desaparezcan** todos los productos de esa categoría (Merluza, Emperador...) de la base de datos, para no dejar productos huérfanos.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_5;
CREATE DATABASE actividad2_5;
USE actividad2_5;

-- 1. Primero la tabla PADRE (Independiente)
CREATE TABLE categorias (
    codigo INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    CONSTRAINT pk_categorias PRIMARY KEY (codigo)
);

-- 2. Luego la tabla HIJA (Dependiente)
CREATE TABLE productos (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    precio DECIMAL (4,2) NOT NULL,
    descripcion VARCHAR(50) NOT NULL,
    codigo_categoria INT NOT NULL,

    CONSTRAINT pk_productos PRIMARY KEY (id),

    -- Relación con borrado en cascada
    CONSTRAINT fk_productos_categorias FOREIGN KEY (codigo_categoria)
        REFERENCES categorias (codigo) ON DELETE CASCADE
);

-- Test para probarlo
INSERT INTO categorias (codigo, descripcion) VALUES
(1, 'Pescado'),
(2, 'Fruta');

INSERT INTO productos (nombre, precio, descripcion, codigo_categoria) VALUES
('Manzana', 2.70, 'Golden', 2),
('Merluza', 7.71, 'Atlántico', 1);
```

Fíjate en esto:

1. **ON DELETE CASCADE:** Esta es la instrucción mágica. Si ejecutas `DELETE FROM categorias WHERE descripcion = 'Pescado';`, verás que la Merluza desaparece sola de la tabla productos. ¡Pruébalo!
2. **DECIMAL (4,2):** Cuidado con esto. Significa que el precio máximo es 99.99 (4 dígitos en total, 2 de ellos decimales). Si intentas meter un producto de 100€, dará error. Para precios normales mejor `DECIMAL (10,2)`.

Actividad 2.6: Empleados (Relación Reflexiva)

Enunciado: Crea una tabla de empleados con DNI y Nombre. Cada empleado tiene un **Jefe** (dni_superior).

- El jefe es, a su vez, otro empleado que está en la misma tabla.
- **Importante:** El "Jefazo Supremo" no tiene jefe (no reporta a nadie).

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_6;
CREATE DATABASE actividad2_6;
USE actividad2_6;

CREATE TABLE empleados (
    dni CHAR(9) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    dni_superior CHAR(9) NULL, -- Ojo: Debe permitir NULL

    CONSTRAINT pk_empleados PRIMARY KEY (dni),

    -- La FK apunta a la PROPIA tabla
    CONSTRAINT fk_empleado_superior FOREIGN KEY (dni_superior)
        REFERENCES empleados (dni)
);

-- Test: Insertamos la jerarquía
INSERT INTO empleados (dni, nombre, dni_superior) VALUES
('01000000J', 'El Jefazo', NULL),          -- No tiene jefe
('02000000I', 'Encargado', '01000000J'),   -- Su jefe es el Jefazo
('03000000E', 'Currito 1', '02000000I'), -- Su jefe es el Encargado
('04000000E', 'Currito 2', '02000000I');
```

Fíjate en esto:

1. **dni_superior CHAR(9) NULL: Obligatorio** que sea NULL. Si lo pones NOT NULL, tendrás el problema del "huevo y la gallina": no podrás insertar al primer empleado porque te pedirá un jefe que aún no existe.
2. **REFERENCES empleados (dni):** La clave foránea apunta a la misma tabla. Es un "bucle"

Actividad 2.11: Compras (Relación N:M)

Enunciado: Vamos a modelar un sistema de compras.

1. Tenemos **Clientes** y **Productos**.
2. Tenemos **Proveedores** que suministran los productos (1:N).
3. **Compras:** Un cliente compra un producto. Como un cliente puede comprar varios y un producto venderse a varios, necesitamos una **tabla intermedia** que conecte los dos mundos.

Copia y pega en tu MySQL: *(He corregido un pequeño error ortográfico del original: "provedores" -> "proveedores")*

SQL

```
DROP DATABASE IF EXISTS actividad2_11;
CREATE DATABASE actividad2_11;
USE actividad2_11;

-- 1. Tablas Independientes (Proveedores y Clientes)
CREATE TABLE clientes (
    DNI CHAR(9) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    CONSTRAINT pk_clientes PRIMARY KEY (DNI)
);

CREATE TABLE proveedores (
    CIF CHAR(10) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    CONSTRAINT pk_proveedores PRIMARY KEY (CIF)
);

-- 2. Tabla con FK simple (Productos)
CREATE TABLE productos (
    codigo DECIMAL(6,0) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    precio DECIMAL(5,0) NOT NULL, -- Ojo, sin decimales según el script original
    CIF_proveedor CHAR(10) NOT NULL,

    CONSTRAINT pk_productos PRIMARY KEY (codigo),
    -- Un producto lo trae un proveedor
    CONSTRAINT fk_productos_proveedores FOREIGN KEY (CIF_proveedor)
        REFERENCES proveedores (CIF)
);

-- 3. LA CLAVE: Tabla Intermedia (Compras)
CREATE TABLE compras (
    DNI_cliente CHAR(9) NOT NULL,
    codigo_producto DECIMAL(6,0) NOT NULL,

    -- PK Compuesta: La combinación Cliente+Producto no se puede repetir
    CONSTRAINT pk_compras PRIMARY KEY (DNI_cliente, codigo_producto),

    -- Dos FKs: Una a cada "padre"
    CONSTRAINT fk_compras_clientes FOREIGN KEY (DNI_cliente)
        REFERENCES clientes (DNI),
    CONSTRAINT fk_compras_productos FOREIGN KEY (codigo_producto)
        REFERENCES productos (codigo)
);
```

```
-- Datos de prueba
INSERT INTO clientes VALUES ('000000001R', 'Juan', 'Pérez', 'Calle A', '2000-01-01');
INSERT INTO proveedores VALUES ('A000000001', 'Sony', 'Japón');
INSERT INTO productos VALUES (1, 'PlayStation 5', 500, 'A000000001');

-- Juan compra la PS5
INSERT INTO compras VALUES ('000000001R', 1);
```

Fíjate en esto:

1. **Tabla Intermedia *compras*:** No tiene datos propios (como nombre o dirección), solo guarda IDs de otras tablas. Su único trabajo es relacionar.
2. **PRIMARY KEY (DNI_cliente, codigo_producto):**
 - **Lo bueno:** Evita duplicados. Juan no puede comprar el producto "1" dos veces.
 - **La trampa del examen:** Si te piden que Juan pueda comprar lo mismo en días diferentes, **esto fallaría**. Tendrías que añadir fecha a la Primary Key (como veremos en el ejercicio 2.12).

Actividad 2.12: Furgonetas (Clave Primaria de 3 Columnas)

Enunciado: Una empresa de reparto quiere controlar qué furgoneta conduce cada repartidor.

- Un repartidor puede conducir **varias furgonetas** a lo largo del tiempo.
- Una furgoneta es conducida por **varios repartidores** en distintas fechas.
- Necesitamos saber **CUÁNDO** (fecha) cogió Juan la furgoneta "1234-BBB". Si no guardamos la fecha, Juan solo podría coger esa furgoneta una vez en su vida.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_12;
CREATE DATABASE actividad2_12;
USE actividad2_12;

-- 1. Tablas Geográficas (Auxiliares)
CREATE TABLE provincias (
    codigo NUMERIC(2,0) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    CONSTRAINT pk_provincias PRIMARY KEY (codigo)
);

CREATE TABLE poblaciones (
    codigo INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    codigo_provincia NUMERIC(2,0) NOT NULL,
    CONSTRAINT pk_poblaciones PRIMARY KEY (codigo),
    CONSTRAINT fk_poblaciones_provincias FOREIGN KEY (codigo_provincia)
        REFERENCES provincias (codigo)
);

-- 2. Tablas Principales
CREATE TABLE repartidores (
    DNI CHAR(9) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    codigo_poblacion INT NOT NULL,
    salario NUMERIC(7,2) NOT NULL,
    CONSTRAINT pk_repartidores PRIMARY KEY (DNI),
    CONSTRAINT fk_repartidores_poblaciones FOREIGN KEY (codigo_poblacion)
        REFERENCES poblaciones (codigo)
);

CREATE TABLE furgonetas (
    matricula CHAR(7) NOT NULL,
    marca VARCHAR(50) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    CONSTRAINT pk_furgonetas PRIMARY KEY (matricula)
);

-- 3. EL NÚCLEO: La asignación con Histórico
CREATE TABLE furgonetas_asignadas (
    DNI_repartidor CHAR(9) NOT NULL,
    matricula_furgoneta CHAR(7) NOT NULL,
    fecha DATE NOT NULL,

    -- ¡ATENCIÓN! La PK son 3 columnas
    CONSTRAINT pk_furgonetas_asignadas PRIMARY KEY (DNI_repartidor,
matricula_furgoneta, fecha),

    CONSTRAINT fk_asignadas_repartidores FOREIGN KEY (DNI_repartidor)
```

```

        REFERENCES repartidores (DNI),
        CONSTRAINT fk_asignadas_furgonetas FOREIGN KEY (matricula_furgoneta)
        REFERENCES furgonetas (matricula)
    );

-- 4. Paquetes (Para completar el ejercicio)
CREATE TABLE paquetes (
    codigo INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(100) NOT NULL,
    nombre_destinatario VARCHAR(100) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    codigo_poblacion INT NOT NULL,
    DNI_repartidor CHAR(9) NOT NULL,
    CONSTRAINT pk_paquetes PRIMARY KEY (codigo),
    CONSTRAINT fk_paquetes_poblaciones FOREIGN KEY (codigo_poblacion)
    REFERENCES poblaciones (codigo),
    CONSTRAINT fk_paquetes_repartidores FOREIGN KEY (DNI_repartidor)
    REFERENCES repartidores (DNI)
);

```

Fíjate en esto:

1. **PRIMARY KEY (DNI_repartidor, matricula_furgoneta, fecha):**
 - Al incluir la **fecha**, permites que la combinación (Juan + Furgoneta) se repita, siempre que sea en días distintos.
 - Si quitaras la fecha, Juan solo podría conducir esa furgoneta una vez y nunca más.
2. **NUMERIC vs INT:** En códigos postales o de provincias (como el '28' de Madrid) se suele usar **NUMERIC (2)** o **CHAR (2)** porque no vamos a sumar ni restar esos números.

Actividad 2.13: Instituto (Booleanos y Claves Únicas)

Enunciado: Diseña la base de datos de un instituto.

1. **Profesores:** Tienen un teléfono que **no se puede repetir** (UNIQUE).
2. **Módulos** (Asignaturas): Cada módulo lo imparte un profesor (1:N).
3. **Clases:** Grupos de alumnos (ej. "1DAM").
4. **Alumnos:** Pertenecen a una clase. Necesitamos saber si un alumno es **delegado o no** (Verdadero/Falso).
5. **Matrículas:** Un alumno se matricula en muchos módulos, y un módulo tiene muchos alumnos (N:M).

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_13;
CREATE DATABASE actividad2_13;
USE actividad2_13;

-- 1. Tablas Independientes (Profesores y Clases)
CREATE TABLE profesores (
    codigo INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    DNI CHAR(9) NOT NULL,
    telefono CHAR(9) NOT NULL UNIQUE, -- Un teléfono no puede estar dos veces
    CONSTRAINT pk_profesores PRIMARY KEY (codigo)
);

CREATE TABLE clases (
    codigo CHAR(6) NOT NULL, -- Ej: '1DAM'
    CONSTRAINT pk_clases PRIMARY KEY (codigo)
);

-- 2. Tablas Hijas (Módulos y Alumnos)
CREATE TABLE modulos (
    codigo INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    codigo_profesor INT NOT NULL,
    CONSTRAINT pk_modulos PRIMARY KEY (codigo),
    CONSTRAINT fk_modulos_profesores FOREIGN KEY (codigo_profesor)
        REFERENCES profesores (codigo)
);

CREATE TABLE alumnos (
    numero_expediente NUMERIC(6,0) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    es_delegado BOOLEAN NOT NULL, -- Nuevo tipo de dato
    codigo_clase CHAR(6) NOT NULL,

    CONSTRAINT pk_alumnos PRIMARY KEY (numero_expediente),
    CONSTRAINT fk_alumnos_clases FOREIGN KEY (codigo_clase)
        REFERENCES clases (codigo)
);

-- 3. Tabla Intermedia (Matrículas N:M)
CREATE TABLE matriculas (
    codigo_modulo INT NOT NULL,
    numero_expediente_alumno NUMERIC(6,0) NOT NULL,
```

```

    CONSTRAINT pk_matriculas PRIMARY KEY (codigo_modulo,
numero_expediente_alumno),

    CONSTRAINT fk_matriculas_modulos FOREIGN KEY (codigo_modulo)
        REFERENCES modulos (codigo),
    CONSTRAINT fk_matriculas_alumnos FOREIGN KEY (numero_expediente_alumno)
        REFERENCES alumnos (numero_expediente)
);

-- Test
INSERT INTO profesores VALUES (1, 'Juan Profe', '00000001A', '666111222');
INSERT INTO clases VALUES ('1DAM');
INSERT INTO alumnos VALUES (100, 'Pepe', 'Alumno', '2005-01-01', TRUE, '1DAM');
-- Fíjate en el TRUE para el booleano

```

Fíjate en esto:

1. **BOOLEAN:** Se usa para campos de Sí/No (como es_delegado). En MySQL, puedes insertar TRUE o FALSE (o 1 y 0).
2. **UNIQUE en Teléfono:** Es diferente a la Primary Key. El profesor se identifica por su código (1, 2, 3), pero bloqueamos que dos profesores tengan el mismo número de móvil con UNIQUE.
3. **PK en Matrículas:** Al poner (codigo_modulo, numero_expediente) como PK, evitas que un alumno se matricule dos veces de la misma asignatura por error.

Actividad 2.14: Tienda Informática (Doble N:M y Ventas)

Enunciado: Diseña la base de datos de una tienda.

1. **Productos:** Tienen existencias (stock).
2. **Proveedores:** Suministran productos. La relación es **Provisiones**.
 - *Detalle:* Aquí la clave primaria es solo (producto, proveedor). Asumimos que define "quién me vende qué".
3. **Clientes:** Compran productos. La relación es **Ventas**.
 - *Detalle:* Un cliente puede comprar el mismo producto varias veces en fechas distintas. Por tanto, la **Fecha** debe formar parte de la clave primaria.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_14;
CREATE DATABASE actividad2_14;
USE actividad2_14;

-- 1. Tablas Maestras (Independientes)
CREATE TABLE clientes (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    telefono CHAR(9) NOT NULL UNIQUE,
    CONSTRAINT pk_clientes PRIMARY KEY (id)
);

CREATE TABLE proveedores (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    CIF CHAR(10) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    telefono CHAR(9) NOT NULL UNIQUE,
    CONSTRAINT pk_proveedores PRIMARY KEY (id)
);

-- 2. Tabla Central
CREATE TABLE productos (
    id INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(500) NOT NULL,
    precio DECIMAL (5,0) NOT NULL,
    existencias DECIMAL (4,0) NOT NULL,
    CONSTRAINT pk_productos PRIMARY KEY (id)
);

-- 3. Relación de Entrada (Provisiones)
CREATE TABLE provisiones (
    id_producto INT NOT NULL,
    id_proveedor INT NOT NULL,
    fecha DATE NOT NULL,

    -- PK Estándar: Un proveedor suministra un producto (definición de catálogo)
    CONSTRAINT pk_provisiones PRIMARY KEY (id_producto, id_proveedor),

    CONSTRAINT fk_provisiones_productos FOREIGN KEY (id_producto)
        REFERENCES productos (id),
    CONSTRAINT fk_provisiones_proveedores FOREIGN KEY (id_proveedor)
        REFERENCES proveedores (id)
```

```

);

-- 4. Relación de Salida (Ventas)
CREATE TABLE ventas (
    id_producto INT NOT NULL,
    id_cliente INT NOT NULL,
    fecha DATE NOT NULL,

    -- PK Histórica: Incluye la FECHA para permitir compras repetidas
    CONSTRAINT pk_ventas PRIMARY KEY (id_producto, id_cliente, fecha),

    CONSTRAINT fk_ventas_productos FOREIGN KEY (id_producto)
        REFERENCES productos (id),
    CONSTRAINT fk_ventas_clientes FOREIGN KEY (id_cliente)
        REFERENCES clientes (id)
);

-- Test rápido
INSERT INTO productos VALUES (1, 'Ratón USB', 10, 50);
INSERT INTO clientes VALUES (1, 'Ana', 'García', 'Calle Pez', '666777888');
-- Ana compra el ratón hoy
INSERT INTO ventas VALUES (1, 1, '2023-10-01');
-- Ana vuelve a comprar el ratón mañana (Permitido gracias a la fecha en la PK)
INSERT INTO ventas VALUES (1, 1, '2023-10-02');

```

Fíjate en esto:

1. Diferencia de Primary Keys:

- En provisiones la PK es (producto, proveedor).
- En ventas la PK es (producto, cliente, fecha).
- *Pregunta de examen:* "¿Por qué en Ventas está la fecha y en Provisiones no?"
- *Respuesta:* "En Ventas registramos cada ticket de compra (histórico). En Provisiones (según este diseño) registramos el catálogo: 'Sony me provee PlayStations', y ese hecho es único, aunque me traigan stock muchas veces".

2. **Tablas pivote:** Tanto ventas como provisiones son tablas pivote o intermedias que conectan la tabla central productos con el mundo exterior.

Actividad 2.15: Liga de Fútbol (Doble FK a la misma tabla)

Enunciado: Diseña la base de datos de una liga.

1. **Presidentes:** Dirigen equipos.
2. **Equipos:** Tienen un presidente (1:1 o 1:N según diseño, aquí usaremos 1:N simplificado).
3. **Jugadores:** Juegan en un equipo y tienen una posición.
4. **Partidos:** Aquí está la clave. Un partido enfrenta a un **Equipo Local** y a un **Equipo Visitante**. Ambos son equipos.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_15;
CREATE DATABASE actividad2_15;
USE actividad2_15;

-- 1. Tablas Auxiliares
CREATE TABLE posiciones (
    id INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    CONSTRAINT pk_posiciones PRIMARY KEY (id)
);

CREATE TABLE presidentes (
    id INT NOT NULL AUTO_INCREMENT,
    DNI CHAR(9) NOT NULL UNIQUE,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    anio_elegido YEAR NOT NULL,
    CONSTRAINT pk_presidentes PRIMARY KEY (id)
);

-- 2. Equipos (Relación con Presidente)
CREATE TABLE equipos (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    nombre_estadio VARCHAR(50) NOT NULL,
    ciudad VARCHAR(50) NOT NULL,
    aforo NUMERIC(6,0) NOT NULL,
    anio_fundacion YEAR NOT NULL,
    id_presidente INT NOT NULL,

    CONSTRAINT pk_equipos PRIMARY KEY (id),
    CONSTRAINT fk_equipos_presidentes FOREIGN KEY (id_presidente)
        REFERENCES presidentes (id)
);

-- 3. Jugadores (Relación con Equipo y Posición)
CREATE TABLE jugadores (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    id_equipo INT NOT NULL,
    id_posicion INT NOT NULL,

    CONSTRAINT pk_jugadores PRIMARY KEY (id),
    CONSTRAINT fk_jugadores_equipos FOREIGN KEY (id_equipo)
        REFERENCES equipos (id),
    CONSTRAINT fk_jugadores_posiciones FOREIGN KEY (id_posicion)
```

```

REFERENCES posiciones (id)
);

-- 4. EL NÚCLEO: Partidos (Doble relación con Equipos)
CREATE TABLE partidos (
    id INT NOT NULL AUTO_INCREMENT,
    fecha DATE NOT NULL,
    goles_locales INT NOT NULL DEFAULT 0,
    goles_visitantes INT NOT NULL DEFAULT 0,

    -- Necesitamos DOS columnas de ID
    id_equipo_local INT NOT NULL,
    id_equipo_visitante INT NOT NULL,

    CONSTRAINT pk_partidos PRIMARY KEY (id),

    -- FK 1: Apunta a equipos (para el local)
    CONSTRAINT fk_partidos_local FOREIGN KEY (id_equipo_local)
        REFERENCES equipos (id),

    -- FK 2: Apunta TAMBIÉN a equipos (para el visitante)
    CONSTRAINT fk_partidos_visitante FOREIGN KEY (id_equipo_visitante)
        REFERENCES equipos (id),

    -- Validación lógica: Un equipo no puede jugar contra sí mismo
    CONSTRAINT chk_rivales_distintos CHECK (id_equipo_local <>
id_equipo_visitante)
);

-- Test
INSERT INTO posiciones (descripcion) VALUES ('Portero'), ('Delantero');
INSERT INTO presidentes VALUES (1, '11111111A', 'Presi', 'Uno', '1980-01-01',
2020);
INSERT INTO equipos VALUES (1, 'Real Madrid', 'Bernabéu', 'Madrid', 81000, 1902,
1);
INSERT INTO equipos VALUES (2, 'Barcelona', 'Camp Nou', 'Barcelona', 99000,
1899, 1);

-- Partido Madrid vs Barça
INSERT INTO partidos (fecha, id_equipo_local, id_equipo_visitante) VALUES
('2023-10-28', 1, 2);

```

Fíjate en esto:

1. Doble FK en Partidos:

- `id_equipo_local -> equipos(id)`
- `id_equipo_visitante -> equipos(id)`
- Es fundamental llamar a las FKs con nombres distintos (`fk_partidos_local` y `fk_partidos_visitante`).

2. CHECK `id_equipo_local <> id_equipo_visitante`: Para evitar que alguien inserte un partido "Madrid vs Madrid".

3. YEAR: Un tipo de dato muy cómodo para guardar años (como fundación o elección) sin usar fechas completas.

Actividad 2.16: La Familia y la Empresa (Relaciones Cruzadas)

Enunciado: Una empresa quiere controlar a sus empleados y sus familias (para temas de guardería y ayudas).

1. **Centros de Trabajo y Categorías:** Tablas auxiliares simples.
2. **Departamentos:** Están ubicados en un centro de trabajo.
3. **Empleados:**
 - Trabajan en un departamento (asignado).
 - Pueden ser jefes de un departamento (dirige). **Nota:** Aquí un empleado apunta dos veces a la tabla departamentos (una como "obrero" y otra como "jefe").
4. **Hijos:**
 - Tienen un **Padre** y una **Madre**, ambos empleados.
 - Necesitamos **dos claves foráneas** a la tabla empleados.
 - Debemos evitar que el padre y la madre sean el mismo empleado (CHECK).

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_16;
CREATE DATABASE actividad2_16;
USE actividad2_16;

-- 1. Tablas Auxiliares
CREATE TABLE categorias (
    id INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    CONSTRAINT pk_categorias PRIMARY KEY (id)
);

CREATE TABLE centros_trabajo (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    CONSTRAINT pk_centros_trabajo PRIMARY KEY (id)
);

-- 2. Departamentos (Padre de Empleados)
CREATE TABLE departamentos (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    presupuesto NUMERIC(8,2) NOT NULL,
    id_centro_trabajo INT NOT NULL,
    CONSTRAINT pk_departamentos PRIMARY KEY (id),
    CONSTRAINT fk_dept_centro FOREIGN KEY (id_centro_trabajo)
        REFERENCES centros_trabajo (id)
);

-- 3. Empleados (Doble relación con Departamentos)
CREATE TABLE empleados (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    telefono CHAR(9) NOT NULL UNIQUE,
    fecha_alta DATE NOT NULL,
    salario NUMERIC(8,2) NOT NULL,
    id_categoria INT NOT NULL,
```

```

-- Dos columnas que apuntan a Departamentos
id_departamento_asignado INT NOT NULL,
id_departamento_dirige INT NULL, -- Puede no dirigir ninguno

CONSTRAINT pk_empleados PRIMARY KEY (id),

CONSTRAINT fk_empleados_cat FOREIGN KEY (id_categoria)
    REFERENCES categorias (id),

-- Relación 1: Trabaja en...
CONSTRAINT fk_empleados_dept_asignado FOREIGN KEY (id_departamento_asignado)
    REFERENCES departamentos (id),

-- Relación 2: Es jefe de...
CONSTRAINT fk_empleados_dept_dirige FOREIGN KEY (id_departamento_dirige)
    REFERENCES departamentos (id)
);

-- 4. Hijos (Doble relación con Empleados + CHECK)
CREATE TABLE hijos (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE NOT NULL,

    -- Dos columnas que apuntan a Empleados
    id_empleado_padre INT NULL, -- Podría ser solo madre soltera
    id_empleado_madre INT NULL,

    CONSTRAINT pk_hijos PRIMARY KEY (id),

    -- FK al Padre
    CONSTRAINT fk_hijos_padre FOREIGN KEY (id_empleado_padre)
        REFERENCES empleados (id),

    -- FK a la Madre
    CONSTRAINT fk_hijos_madre FOREIGN KEY (id_empleado_madre)
        REFERENCES empleados (id),

    -- VALIDACIÓN: Padre y Madre no pueden ser la misma persona
    CONSTRAINT chk_padres_distintos CHECK (id_empleado_padre <>
id_empleado_madre)
);

-- Test
INSERT INTO categorias VALUES (1, 'Ingeniero');
INSERT INTO centros_trabajo VALUES (1, 'Oficina Central', 'Madrid');
INSERT INTO departamentos VALUES (1, 'I+D', 50000, 1);

-- Insertamos Padre y Madre
INSERT INTO empleados VALUES (1, 'Padre', 'Perez', '666111222', '2020-01-01',
2000, 1, 1, NULL);
INSERT INTO empleados VALUES (2, 'Madre', 'Gomez', '666333444', '2020-01-01',
2000, 1, 1, 1); -- Ella dirige el Dept 1

-- Insertamos Hijo (Correcto)
INSERT INTO hijos VALUES (1, 'Niño', '2023-01-01', 1, 2);

-- Insertamos Hijo (Incorrecto - Error CHECK)
-- INSERT INTO hijos VALUES (2, 'Clon', '2023-01-01', 1, 1);

```

Fíjate en esto:

1. **Doble FK en empleados:** Un empleado tiene dos "vínculos" con departamentos. Uno obligatorio (*asignado*, donde trabaja) y otro opcional (*dirige*, si es jefe).
2. **CHECK (id_padre <> id_madre):** Es fundamental. Sin esto, la base de datos permitiría biológicamente imposibles (que alguien sea padre y madre a la vez del mismo niño).
3. **NULL en los padres:** Hemos puesto NULL porque puede que solo trabaje en la empresa uno de los dos progenitores.

Actividad 2.17: Cadena de Hoteles (El Check XOR)

Enunciado: Diseña la base de datos para una cadena hotelera.

1. **Clientes:** Deben identificarse obligatoriamente, pero pueden elegir entre usar DNI o Pasaporte. No pueden estar los dos vacíos ni los dos llenos.
2. **Hoteles:** Tienen categorías (estrellas).
3. **Habitaciones:** Pertenecen a un hotel.
4. **Reservas:** Un cliente reserva una habitación en unas fechas. Aquí la **Fecha** es parte de la clave para que la habitación no se reserve dos veces el mismo día.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_17;
CREATE DATABASE actividad2_17;
USE actividad2_17;

-- 1. Tablas Auxiliares
CREATE TABLE categorias (
    id INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    CONSTRAINT pk_categorias PRIMARY KEY (id)
);

CREATE TABLE tipo_habitacion (
    id INT NOT NULL AUTO_INCREMENT,
    descripcion VARCHAR(50) NOT NULL,
    CONSTRAINT pk_tipo_habitacion PRIMARY KEY (id)
);

-- 2. Clientes con el famoso CHECK XOR
CREATE TABLE clientes (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(150) NOT NULL,
    dni CHAR(9) NULL,          -- Puede ser nulo
    pasaporte CHAR(9) NULL,   -- Puede ser nulo
    telefono CHAR(9) NOT NULL UNIQUE,

    CONSTRAINT pk_clientes PRIMARY KEY (id),

    -- LA CLAVE DEL EJERCICIO:
    -- (Tiene DNI y NO Pasaporte) O (Tiene Pasaporte y NO DNI)
    CONSTRAINT chk_dni_pasaporte CHECK (
        (dni IS NOT NULL AND pasaporte IS NULL)
        OR
        (dni IS NULL AND pasaporte IS NOT NULL)
    )
);

-- 3. Hoteles
CREATE TABLE hoteles (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    direccion VARCHAR(200) NOT NULL,
    telefono CHAR(9) NOT NULL,
    id_categoria INT NOT NULL,
    CONSTRAINT pk_hoteles PRIMARY KEY (id),
    CONSTRAINT fk_hoteles_categoria FOREIGN KEY (id_categoria)
        REFERENCES categorias (id)
);
```

```

-- 4. Habitaciones
CREATE TABLE habitaciones (
    id INT NOT NULL AUTO_INCREMENT,
    numero_habitacion NUMERIC(4,0) NOT NULL,
    id_hotel INT NOT NULL, -- Falta vincularla al hotel
    id_tipo INT NOT NULL, -- Simple/Doble

    CONSTRAINT pk_habitaciones PRIMARY KEY (id),
    CONSTRAINT fk_habitaciones_hoteles FOREIGN KEY (id_hotel)
        REFERENCES hoteles (id),
    CONSTRAINT fk_habitaciones_tipo FOREIGN KEY (id_tipo)
        REFERENCES tipo_habitacion (id)
);

-- 5. Reservas (Histórico)
CREATE TABLE reservas (
    id INT NOT NULL AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    id_habitacion INT NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    precio DECIMAL(6,2) NOT NULL,

    CONSTRAINT pk_reservas PRIMARY KEY (id),

    CONSTRAINT fk_reservas_clientes FOREIGN KEY (id_cliente)
        REFERENCES clientes (id),
    CONSTRAINT fk_reservas_habitaciones FOREIGN KEY (id_habitacion)
        REFERENCES habitaciones (id)
);

-- Test del XOR
-- Esto funciona (Solo DNI)
INSERT INTO clientes (nombre, dni, pasaporte, telefono) VALUES ('Pepe',
'12345678A', NULL, '666111000');
-- Esto funciona (Solo Pasaporte)
INSERT INTO clientes (nombre, dni, pasaporte, telefono) VALUES ('John', NULL,
'PASSPORT1', '666222000');
-- Esto FALLA (Los dos a la vez)
-- INSERT INTO clientes (nombre, dni, pasaporte, telefono) VALUES ('Fail',
'12345678A', 'PASSPORT1', '666333000');

```

Fíjate en esto:

1. La Lógica XOR:

- dni IS NOT NULL AND pasaporte IS NULL -> Tengo DNI.
- OR -> O bien...
- dni IS NULL AND pasaporte IS NOT NULL -> Tengo Pasaporte.
- Cualquier otra combinación (los dos null o los dos llenos) dará falso y el CHECK bloqueará la inserción.

2. **Campos NULL:** Para que esto funcione, es vital que defines dni y pasaporte como NULL (opcionales) al crear la tabla. Si pones NOT NULL, obligarías a rellenar siempre los dos.

Enunciado: Diseña la base de datos para el circuito profesional de tenis.

1. **Ediciones de Torneos:** Un torneo (ej. "Roland Garros") se celebra cada año.

- No basta con el ID del torneo. Una **Edición** se identifica por (Año + ID_Torneo).
- Ejemplo: La edición de 2024 de Roland Garros es distinta a la de 2023.

2. **Partidos:** Son el centro de todo.

- Pertenecen a una **Edición** concreta (necesitamos FK compuesta: año y torneo).
- Tienen **Fase** (Final, Semifinal...), **Modalidad** (Individual/Dobles), **Árbitro**, **Ciudad**...
- **Jugadores:** Un partido de dobles enfrenta a 4 personas. Necesitamos **4 columnas** de jugador (Jugador 1 y su pareja vs Jugador 2 y su pareja).
- *Nota:* En este script estricto, todos los campos de jugadores son obligatorios (NOT NULL), lo que nos obligaría a registrar siempre 4 jugadores (ideal para un torneo solo de dobles). En la vida real, las parejas serían NULL para partidos individuales.

Copia y pega en tu MySQL:

SQL

```
DROP DATABASE IF EXISTS actividad2_18;
CREATE DATABASE actividad2_18;
USE actividad2_18;

-- 1. Tablas Auxiliares Básicas
CREATE TABLE arbitros (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    CONSTRAINT pk_arbitros PRIMARY KEY (id)
);

CREATE TABLE torneos (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    CONSTRAINT pk_torneos PRIMARY KEY (id)
);

CREATE TABLE fases (
    id INT NOT NULL AUTO_INCREMENT,
    fase VARCHAR(50) NOT NULL, -- Ej: Octavos, Final
    CONSTRAINT pk_fases PRIMARY KEY (id)
);

CREATE TABLE modalidad (
    id INT NOT NULL AUTO_INCREMENT,
    modalidad VARCHAR(50) NOT NULL, -- Ej: Individual Masc, Dobles Mixto
    CONSTRAINT pk_modalidad PRIMARY KEY (id)
);

CREATE TABLE paises (
    id INT NOT NULL AUTO_INCREMENT,
    pais VARCHAR(50) NOT NULL,
    CONSTRAINT pk_paises PRIMARY KEY (id)
);

-- 2. Tablas Intermedias (Dependientes)
CREATE TABLE ciudades (
    id INT NOT NULL AUTO_INCREMENT,
```

```

ciudad VARCHAR(50) NOT NULL,
id_pais INT NOT NULL,
CONSTRAINT pk_ciudades PRIMARY KEY (id),
CONSTRAINT fk_ciudades_paises FOREIGN KEY (id_pais)
    REFERENCES paises (id)
);

CREATE TABLE jugadores (
    id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    id_pais INT NOT NULL,
    CONSTRAINT pk_jugadores PRIMARY KEY (id),
    CONSTRAINT fk_jugadores_paises FOREIGN KEY (id_pais)
        REFERENCES paises (id)
);

-- 3. Entidad Débil: EDICIÓN (Clave Compuesta)
CREATE TABLE edicion (
    anio YEAR NOT NULL,
    id_torneo INT NOT NULL,

    -- La PK es la combinación de AÑO + TORNEO
    CONSTRAINT pk_edicion PRIMARY KEY (anio, id_torneo),

    CONSTRAINT fk_edicion_torneos FOREIGN KEY (id_torneo)
        REFERENCES torneos (id)
);

-- 4. EL MONSTRUO FINAL: Partidos
CREATE TABLE partidos (
    id INT NOT NULL AUTO_INCREMENT,
    resultado VARCHAR(10) NOT NULL,          -- Ej: '6-4 6-3'
    premio_ganador NUMERIC(6,0) NULL,
    premio_consolacion NUMERIC(6,0) NULL,

    -- FK Compuesta hacia Edición (Año + Torneo)
    id_edicion_anio YEAR NOT NULL,
    id_torneo INT NOT NULL,

    -- FKs Simples
    id_ciudad INT NOT NULL,
    id_fase INT NOT NULL,
    id_modalidad INT NOT NULL,
    id_arbitro INT NOT NULL,

    -- Los 4 Jugadores (Dobles)
    id_jugador1 INT NOT NULL,
    id_jugador1_doble INT NOT NULL,
    id_jugador2 INT NOT NULL,
    id_jugador2_doble INT NOT NULL,

    CONSTRAINT pk_partidos PRIMARY KEY (id),

    -- FK Gigante: Apunta a las dos columnas de la PK de Edición
    CONSTRAINT fk_partidos_edicion FOREIGN KEY (id_edicion_anio, id_torneo)
        REFERENCES edicion (anio, id_torneo),

    CONSTRAINT fk_ciudades_partidos FOREIGN KEY (id_ciudad) REFERENCES ciudades
(id),
    CONSTRAINT fk_fases_partidos FOREIGN KEY (id_fase) REFERENCES fases (id),
    CONSTRAINT fk_modalidad_partidos FOREIGN KEY (id_modalidad) REFERENCES
modalidad (id),
    CONSTRAINT fk_arbitros_partidos FOREIGN KEY (id_arbitro) REFERENCES arbitros
(id),

```

```

-- 4 FKs a la misma tabla Jugadores
CONSTRAINT fk_jugador1 FOREIGN KEY (id_jugador1) REFERENCES jugadores (id),
CONSTRAINT fk_jugador1_doble FOREIGN KEY (id_jugador1_doble) REFERENCES
jugadores (id),
CONSTRAINT fk_jugador2 FOREIGN KEY (id_jugador2) REFERENCES jugadores (id),
CONSTRAINT fk_jugador2_doble FOREIGN KEY (id_jugador2_doble) REFERENCES
jugadores (id)
);

```

Fíjate en esto (Puntos Críticos):

1. La FK Compuesta en **Partidos**:

- Mira esto: FOREIGN KEY (id_edicion_anio, id_torneo) REFERENCES edicion (anio, id_torneo).
- Como la tabla `edicion` tiene una clave primaria de dos columnas, quien quiera relacionarse con ella **necesita dos columnas** también. Es obligatorio que coincidan en tipo y orden.

2. Cuádruple relación con **Jugadores**:

- Es lo mismo que vimos en el fútbol (local/visitante) pero elevado al cuadrado. Tienes 4 columnas distintas apuntando a la tabla `jugadores`.

3. **YEAR como parte de PK**: En `edicion`, el año es parte fundamental de lo que identifica a la fila. Sin el año, no sabríamos distinguir el torneo de 2023 del de 2024.